CMIS 3153 · ADVANCED DATABASE SYSTEMS

Mid Exam Study Pack · Lesson 5

Distributed Database Systems

Lesson in simple English · Formula cheat box · Answer recipes · Tutorial answers · Past paper questions 2020/21 – 2023/24 with answers

INSIDE THIS PACK

- How to use this pack
- Lesson part 1: What a distributed database is
- Lesson part 2: Distributing the data and processing it
- Cheat box: Lesson 5 at a glance
- How-to-answer recipes
- Lesson 5 slide exercises (short answers)
- Past paper questions with answers

★ asked in more than one past paper · (25m) marks in the paper · Yellow boxes = past paper questions · Blue boxes = formulas and key notes · Green boxes = given data

Contents

How to use this pack 4

Quick-find table 4

Lesson part 1: What a distributed database is 6

1.1 Definition 6

1.2 Architecture 6

1.3 Why distributed databases? (advantages) 7

1.4 Centralized vs distributed 7

1.5 Properties of a DDB system 8

1.6 The CAP theorem 8

Lesson part 2: Distributing the data and processing it 10

2.1 Fragmentation (splitting a table) 10

2.2 Replication and allocation 10

2.3 Types of distributed database systems 11

2.4 Design issues of distributed databases 12

2.5 Distributed query processing 12

2.6 Data transfer cost and the semijoin (lecture example) 12

2.7 Concurrency control and recovery 13

Cheat box: Lesson 5 at a glance 15

CAP theorem 15

Transparency types 15

Fragmentation in relational algebra 15

Replication levels 16

Homogeneous vs heterogeneous (one line each) 16

Distributed query processing: the 7 stages 16

Concurrency control and commit 16

How-to-answer recipes 18

Recipe 1: Scenario question — "features that make it distributed" and "benefits" ★ 18

Recipe 2: CAP — "which properties to prioritize" / "trade-offs" ★ 18

Recipe 3: Fragmentation, replication and allocation strategy for a scenario ★ 18

Recipe 4: Distributed query processing questions 19

Recipe 5: Compare / define questions 19

Lesson 5 slide exercises (short answers) 20

Past paper questions with answers 22

A. Concepts and architecture 22

B. Scenario: features and benefits 23

C. Design issues, system types, fragmentation and replication 24

D. CAP theorem and properties in scenarios ★ 27

E. Distributed query processing and transactions 29

F. Fault tolerance 31

Questions NOT in the mid scope 31

How to use this pack

Distributed databases came up in **3 of the 4 papers** and the share is growing: since 2022/23 it is a whole **Question 6**, usually a **scenario** (a multinational company with branches in several cities) asking for features and benefits, fragmentation and replication, distributed query processing and the **CAP theorem**. There is no tutorial for this lesson, so the past papers and the recipes are the key.

In the exam hall:

1. Find the question type in the Quick-find table below.
2. For a scenario question, use Recipe 1: **always name the cities/branches from the question** in your answer.
3. Match the length to the marks: a **25-mark** part needs **4–6 clear points**, a table, and an example from the scenario.

Signs used in this pack: ★ = asked in more than one past paper. (25m) = marks in the paper. DDB = distributed database. DDBMS = distributed database management system.

Quick-find table

If the question asks about...	Go to
What is a distributed database; how it differs from a centralized one ★	Lesson part 1 · 2022/23 Q6(a), 2023/24 Q7(c)
Distributed database architecture (with a diagram)	Lesson part 1 · 2021/22 Q3(a)
Features that make a setup distributed, and its benefits (scenario)	Recipe 1 · 2023/24 Q6(a)
Transparency, autonomy, reliability and availability	Lesson part 1
CAP theorem ★	Lesson part 1 · Cheat box · 2022/23 Q6(b)
Which CAP properties to prioritize / CAP trade-offs (scenario) ★	Recipe 2 · 2022/23 Q6(c)(ii), 2023/24 Q6(d)
Core properties of a DDB applied to a scenario (logistics)	2022/23 Q6(c)(i)
Design issues of distributed databases	Lesson part 2 · 2021/22 Q3(b)
Homogeneous vs heterogeneous (federated, multidatabase)	Lesson part 2 · 2021/22 Q3(c)
Fragmentation and replication: benefits and drawbacks	Lesson part 2 · 2021/22 Q3(d)
Fragmentation / replication / allocation strategy for a scenario ★	Recipe 3 · 2023/24 Q6(b), Q7(d)(i)
Distributed query processing and its challenges	Lesson part 2 · 2023/24 Q6(c)
Data transfer cost, semijoin calculations	Lesson part 2 · Cheat box
Query processing + transaction management across sites	2023/24 Q7(d)(ii)

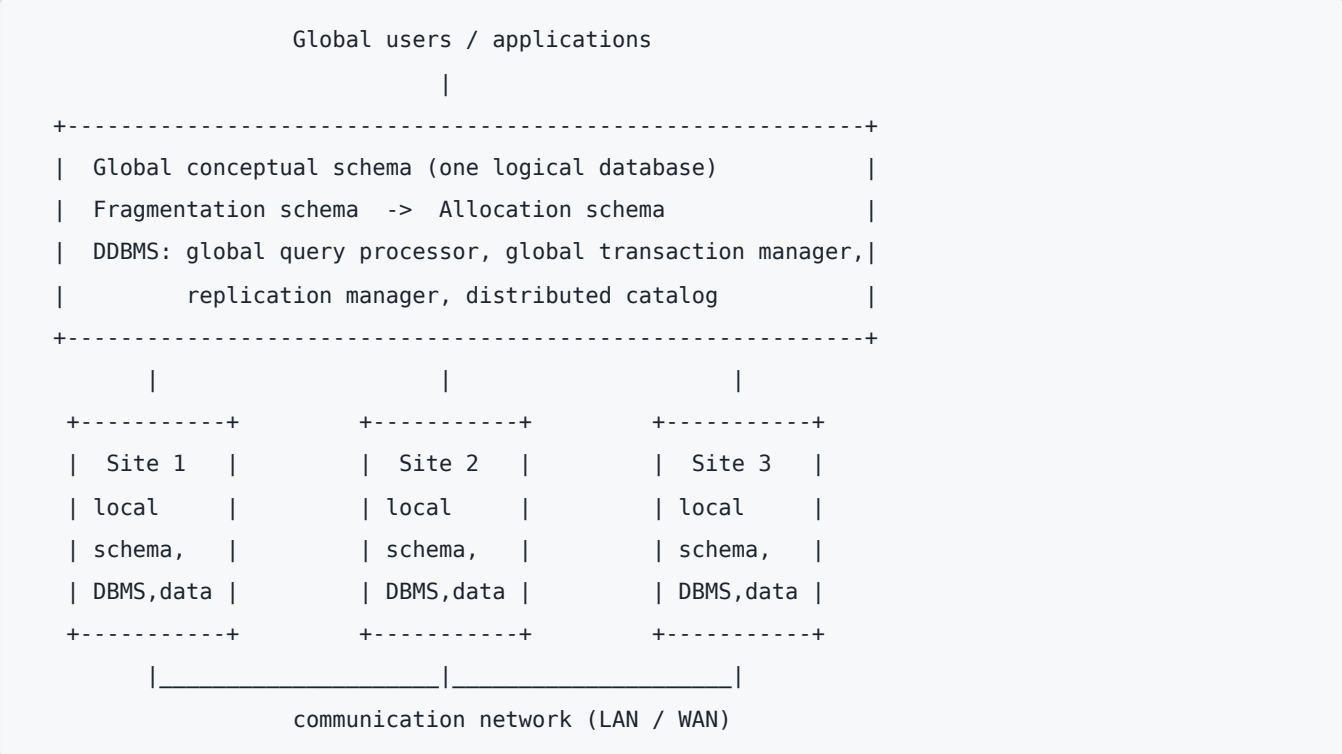
If the question asks about...	Go to
Concurrency control (primary site, primary copy, voting), 2PC, 3PC, recovery	Lesson part 2
Fault tolerance (short note)	2022/23 Q7(f)

Lesson part 1: What a distributed database is

1.1 Definition

- A **distributed database (DDB)** = a **collection of multiple, logically related databases spread over a computer network**.
- A **distributed DBMS (DDBMS)** = the software that **manages the DDB and makes the distribution transparent** (invisible) to users.
- It is made of **loosely coupled sites (nodes)** that share **no physical components** (no shared memory or disks). Each site has its own DBMS, processor and storage.
- A transaction can use data at **one or several sites**, and the sites execute it together in a unified way.
- Users see **one logical database** (a global schema), even though the data is spread out.

1.2 Architecture



- **Sites (nodes):** independent computers, each with a local DBMS and database holding **fragments / copies** of the data.
- **Communication network:** connects the sites; carries queries, data and results.
- **Global conceptual schema:** the single logical view users query.
- **Fragmentation schema:** how each global table is split into fragments.
- **Allocation schema:** which site(s) store each fragment (replication).
- **Local schemas:** how each site stores its part.
- **DDBMS components:** global query processing, distributed transaction management (concurrency control, commit, recovery), replication management, the **distributed catalog (directory)** that records where everything is.

- Common styles: **client–server**, **peer-to-peer (collaborating servers)**, **federated / multidatabase**.

1.3 Why distributed databases? (advantages)

Advantage	Meaning
Scalability	Add more nodes (**horizontal scaling**) instead of buying a bigger server; in the cloud, scale up and down with demand (**elastic**)
High availability / reliability	Data is **replicated**; if one node fails, others continue. Faults are **isolated** to their site; **no single point of failure**
Better performance	Data stored **close to where it is used** (data localization) → less network delay; **parallel** processing of large queries; **load balancing**
Geographic distribution	Users worldwide get low latency; data can stay in its country (**local data laws**)
Disaster recovery	Copies in other locations survive a disaster at one site
Cost efficiency	Many cheap **commodity** machines instead of one expensive server; cloud integration
Easier expansion	Add data, sites or processors without redesigning everything
Easier application development	Transparency hides where data is, so applications are simpler
Big data, real-time, microservices, multi-tenancy, flexible data models	Other uses listed in the lecture

1.4 Centralized vs distributed

Aspect	Centralized database	Distributed database
Data storage	All data at **one site**	Data **fragmented and/or replicated** over several sites, near its users
Processing	One DBMS runs every query	A query is **split into sub-queries** at several sites, run **in parallel**, results **merged**; network cost matters
Transparency	Only logical/physical data independence	Must also hide **location, fragments, copies** (distribution transparency)
Reliability	Single point of failure	A failure affects only its site; copies keep data available
Scalability	Vertical (bigger machine)	Horizontal (more sites)
Transactions	Local concurrency control and recovery	Distributed locking, distributed deadlock, **two-phase commit**
Complexity	Simpler	Complex design (fragmentation, allocation, catalog, consistency)

1.5 Properties of a DDB system

Transparency = hiding the details of distribution from users. Types:

- **Data organization (distribution / network) transparency:** users do not need to know about the network or where data is placed.
  - **Location transparency:** the same command works no matter where the data is or where the command is issued.
  - **Naming transparency:** once an object has a name, it can be used without saying where it is. Every data item needs a **system-wide unique name**; its location can be found efficiently and changed transparently; each site can create new items on its own.
- **Replication transparency:** users do not know there are **copies** of the data at several sites.
- **Fragmentation transparency:** users query a table as if it were **not split**; a global query is turned into fragment queries automatically.
- **Design transparency** and **execution transparency:** users do not need to know how the DDB is designed or where a transaction runs.

Autonomy = how far each site can work **independently**:

- **Design autonomy:** each site chooses its own data model and transaction management techniques.
- **Communication autonomy:** each site decides what information to share with others.
- **Execution autonomy:** each site's users can act as they please (run their own transactions).

Reliability and availability:

- **Reliability** = the probability that the system is **running at a given moment**.
- **Availability** = the probability that the system is **continuously available during a time period**.
- **Fault** → causes an **error** (a bad system state) → which can cause a **failure** (the system does not behave as specified).

Extra functions a DDBMS must provide: keep track of data distribution (expanded catalog); distributed query processing; distributed transaction management; replicated data management (which copy to read, keeping copies consistent); distributed recovery (site crashes, link failures); security across sites; distributed directory (catalog) management.

1.6 The CAP theorem

The **CAP theorem (Brewer's theorem):** a distributed data store can guarantee **at most two** of these three at the same time:

- **C — Consistency:** every read returns the **most recent write**; all nodes see the same data at the same time.
- **A — Availability: every request gets a response** (read or write), even if some nodes are down — though the data might not be the latest.
- **P — Partition tolerance:** the system **keeps working when the network splits** and some nodes cannot talk to others.

Key insight: network partitions **will** happen in a real distributed system, so **P is a must**. So when a partition happens, the system must choose:

- **C (CP):** refuse or delay some requests so no one sees inconsistent data. Examples: **HBase, ZooKeeper**. Good for banking, stock levels, payments.
- **A (AP):** answer every request, even with possibly old data; copies become consistent later (**eventual consistency**). Examples: **Cassandra, Couchbase**. Good for social media feeds, shopping carts, product catalogues.
- **CA:** only **theoretical** for a distributed system (it only works with no partitions, i.e. a single-site database).

Example: two bank branches lose their network link. CP: refuse withdrawals until the link is back (balances stay correct). AP: allow withdrawals using possibly old balances (always available, but the account could be overdrawn twice).

CAP comes from **replication**: copies at several sites must be kept in step over the network. Designers choose CP or AP **per type of data**.

Lesson part 2: Distributing the data and processing it

2.1 Fragmentation (splitting a table)

The database is broken into logical pieces called **fragments**, which are stored at different sites.

Type	What it keeps	In RA	Rebuild the table with	Example
Horizontal	A subset of the **rows**	σ C (R)	**UNION** of the fragments	EMPLOYEE where Dno = 5 stored at department 5's site
Derived horizontal	Rows of a related table, split the **same way** as the main table (through the foreign key)	follows the parent	UNION	EMPLOYEE and PROJECT split by department, like DEPARTMENT
Vertical	A subset of the **columns**, **each fragment must include the primary key**	π L (R)	**OUTER UNION** (or join on the key)	Personal columns (Ssn, Name, Bdate, Address, Sex) and work columns (Ssn, Salary, Super_ssn, Dno)
Mixed (hybrid)	Both	π L (σ C (R))	UNION and OUTER UNION in the right order	Personal columns of department 5's employees

- **Complete horizontal fragmentation:** every row satisfies at least one condition (C1 OR C2 OR ... OR Cn covers all rows). Usually also **disjoint** (no row in two fragments).
- **Complete vertical fragmentation:** the column lists together cover all columns (L1 ∪ L2 ∪ ... = all attributes) and share **only the primary key** (Li ∩ Lj = PK).
- Why the key in every vertical fragment? Without a shared key column the pieces could never be joined back together.
- **Fragmentation schema:** the definition of all fragments (covering all rows and columns, so the database can be rebuilt). **Allocation schema:** which site(s) store each fragment.

2.2 Replication and allocation

- **Replication** = storing **copies** of a fragment at **more than one** site.
- **Full replication:** the whole database at every site.
  - (+) **Very high availability** (works while at least one site is up); **fast reads** (any query answered locally).
  - (−) **Very slow updates** (every copy must be updated); concurrency control and recovery become **much more expensive**.
- **No replication (non-redundant allocation):** each fragment at exactly one site.
- **Partial replication:** some fragments copied, others not (the usual choice). A description of the copies = **replication schema**.
- **Allocation (data distribution)** = deciding which site stores each fragment or copy. Guidance from the lecture:

    - High availability needed, transactions from anywhere, **mostly read-only** → **full replication**.
    - Data used mostly at one site → store it **only at that site**.
    - Data used at several sites → **replicate** it at those sites.
    - **Many updates** → **limit replication**.
- Finding the best allocation is a **complex optimization problem**.

Fragmentation and replication compared with a centralized database:

	Benefits	Drawbacks
Fragmentation	**Data locality** (stored where used) → less network traffic, faster local access; **parallel** queries; smaller tables to scan; local control of sensitive data	Queries needing several fragments need **distributed joins/unions** and network transfer; constraints across fragments are harder; complex design
Replication	**Higher availability and reliability** (no single point of failure); **faster reads** from a nearby copy; load sharing; disaster recovery	**Updates must reach every copy** (slow writes); **consistency**, concurrency control and recovery are harder; more storage; risk of stale data
Centralized DB (for contrast)	Simple, always consistent, no network inside	Single point of failure, slow for remote users, limited scalability

2.3 Types of distributed database systems

They differ in **how similar the software is (homogeneity)** and **how independent each site is (local autonomy)**.

	Homogeneous	Heterogeneous
DBMS software	**Same** at every site (all Oracle, all DB2...); operating systems may differ	**Different** DBMSs and schemas at different sites
Awareness and cooperation	Sites **know each other** and **agree to cooperate**	Sites may **not know each other**; **limited** cooperation
Local autonomy	Lower (sites give up the right to change schema or software freely)	Higher
Query processing	Straightforward	Hard: schema differences must be translated
Transactions across sites	Standard protocols (2PC)	Difficult: software differences
Typical origin	Built as a distributed database from the start	Joining existing / legacy systems

Two kinds of heterogeneous system:

- **Federated database system (FDBS):** different DBMSs, but access goes through **one global conceptual schema**; local autonomy is **minimal**; sites follow a central access policy.

- **Multidatabase system: full local autonomy**, **no global schema**; a schema is built **when needed** by the application.

Sources of heterogeneity in an FDBS: differences in **data models** (relational, object, hierarchical, network, files), in **constraints** (how relationships and rules are expressed, e.g. triggers), and in **query languages** (SQL versions, data types, operators).

2.4 Design issues of distributed databases

1. **Fragmentation:** how to split tables (horizontal, vertical, mixed) so fragments are complete and can be rebuilt.
2. **Allocation:** which site stores each fragment (a complex optimization problem).
3. **Replication:** how many copies and where (availability vs update cost).
4. **Distributed catalog / directory:** tracking fragments, copies and locations; global or local directory.
5. **Distributed query processing:** network cost dominates; choosing strategies (where to run each step, semijoins).
6. **Distributed transactions and concurrency control:** multiple copies, distributed locking, **distributed deadlock**.
7. **Recovery and reliability:** site failures, **link failures**, **network partitioning**, **distributed commit** (2PC/3PC).
8. **Heterogeneity:** different DBMSs, data models, constraints, query languages.
9. **Security** across sites; **autonomy** vs global control.
10. **Consistency vs availability** trade-off (CAP).

2.5 Distributed query processing

The stages (lecture):

1. **Query mapping:** the query is translated into an algebraic query on the **global** tables (using the global schema, ignoring distribution), then normalized, checked for semantic errors, simplified.
2. **Localization:** the global tables are replaced by their **fragments** using the fragmentation and replication information (e.g. Orders → Orders_Colombo ∪ Orders_London ∪ Orders_Singapore).
3. **Global query optimization:** choose the cheapest strategy. Cost = CPU + I/O + **communication**; over a WAN **communication cost is usually the biggest**.
4. **Local query optimization:** each site optimizes its own part (as in a centralized DBMS).
5. **Join processing:** ship the query to the data, ship the data to the query, or a hybrid.
6. **Aggregation:** do **partial aggregation** at each site (local SUM, COUNT) before combining.
7. **Result merging:** results are sent to the result site and merged.

Stages 1–3 run at a central control site; local optimization runs at each site.

2.6 Data transfer cost and the semijoin (lecture example)

EMPLOYEE at site 1: 10,000 rows × 100 bytes = **1,000,000 bytes**. DEPARTMENT at site 2: 100 rows × 35 bytes = **3,500 bytes**. The query is submitted at **site 3** (result site). Result rows are 40 bytes.

Q: each employee's name and department name (10,000 result rows = 400,000 bytes):

Strategy	Bytes transferred
1. Send both tables to site 3, join there	1,000,000 + 3,500 = 1,003,500
2. Send EMPLOYEE to site 2, join, send result to site 3	1,000,000 + 400,000 = 1,400,000
3. Send DEPARTMENT to site 1, join, send result to site 3	3,500 + 400,000 = **403,500** (best)

Q′: each department's name and its manager's name (100 result rows = 4,000 bytes): strategy 1 = 1,003,500; strategy 2 = 1,004,000; strategy 3 = 3,500 + 4,000 = **7,500** (best by far).

Rule: ship the **smaller** table to the bigger one, and ship small results.

Semijoin idea: reduce a table **before** sending it. Send only the **join column** of R to S's site, join it with S there, and send back only the **matching rows of S with the needed columns**. Then do the final join.

For Q′ with the result at site 2: send π Mgr_ssn (DEPARTMENT) to site 1 = 9 × 100 = **900 bytes**; join with EMPLOYEE there and send back the 100 matching managers' (Mgr_ssn, Fname, Lname) = 39 × 100 = **3,900 bytes**; join with DEPARTMENT at site 2. Total = **4,800 bytes** instead of 1,000,000. For Q the semijoin saves little (340,400 bytes) because **every** employee matches a department. **A semijoin helps most when only a small fraction of rows take part in the join.**

2.7 Concurrency control and recovery

Problems that do not exist in a centralized DB:

- **Multiple copies** of data items must be kept consistent; a recovered copy must be brought up to date.
- **Failure of individual sites:** the rest should keep running; a recovered site must catch up before rejoining.
- **Failure of communication links**, including **network partitioning** (sites split into groups that cannot talk to each other).
- **Distributed commit:** a transaction that touches several sites must commit at all of them or none, even if some fail during commit (solved by **two-phase commit**).
- **Distributed deadlock:** a deadlock can span several sites.

Concurrency control with a distinguished copy:

- **Primary site technique:** one site is the **coordinator** for all locking and commits (two-phase locking there guarantees serializability). (+) simple, an extension of centralized 2PL. (–) the primary site gets **overloaded** and is a **single point of failure**; a **backup site** can shadow it and take over.
- **Primary copy technique:** each data item has a **primary copy** at some site; locking an item means locking its primary copy. Primary copies are spread over sites, so no site is overloaded; but finding the primary copy needs a distributed directory.
- **Coordinator failure:** with no backup site → abort and restart all active transactions and **elect** a new coordinator; with a backup site → suspend transactions, the backup becomes primary, choose a new backup, resume; if both fail → **election**.

Concurrency control by voting: no distinguished copy. A lock request goes to **all sites with a copy**; each copy grants or refuses. If a **majority** grants it, the transaction holds the lock; otherwise it cancels after a time-out. Truly distributed, but **more messages**, and complex when sites fail.

Recovery techniques:

- **Logs:** record every change so a site can roll back or redo to a consistent state.
- **Checkpoints:** periodically save the database state so recovery is faster (no need to replay the whole log).
- **Two-phase commit (2PC):** Phase 1 (**prepare / voting**): the coordinator asks every participant to prepare; each writes its log and votes YES (ready) or NO. Phase 2 (**decision**): if **all** vote YES, the coordinator sends COMMIT; otherwise ABORT; participants do it and acknowledge. Result: **all commit or all abort**. Weakness: participants can be **blocked** waiting if the coordinator crashes after they voted YES.
- **Three-phase commit (3PC):** adds an extra **pre-commit** phase so participants can finish the decision even if the coordinator crashes (**non-blocking**).

Cheat box: Lesson 5 at a glance

CAP theorem

Choice	Keeps	Gives up during a partition	Examples (lecture)	Use for
CP	Consistency + Partition tolerance	Availability (some requests refused or delayed)	HBase, ZooKeeper	Bank balances, stock levels, payments, order confirmation
AP	Availability + Partition tolerance	Consistency (stale reads; **eventual consistency**)	Cassandra, Couchbase	Social feeds, shopping carts, catalogues, tracking, dashboards
CA	Consistency + Availability	Partition tolerance	Only theoretical (single-site DB)	–

> **Remember:** P is not optional in a real distributed system, so the real choice is **C or A when the network splits**.

Transparency types

Type	Users do not need to know...
Location	where the data is or where the command runs
Naming	anything except a unique name for each object
Replication	that copies exist
Fragmentation	that a table is split into fragments
Design / execution	how the DDB is designed / where a transaction executes

Autonomy types: design (own data model and techniques), communication (decide what to share), execution (users act independently).

Fragmentation in relational algebra

$$\text{horizontal: } R_i = \sigma_{C_i}(R)$$
$$R = R_1 \cup R_2 \cup \cdots \cup R_n$$

$$\text{vertical: } R_i = \pi_{L_i}(R), \quad PK \subseteq L_i$$
$$R = \text{outer union (or join on PK) of the } R_i$$

$$\text{mixed: } \pi_L(\sigma_C(R))$$

Complete horizontal: C1 OR ... OR Cn covers every row (often disjoint). Complete vertical: L1 ∪ ... ∪ Ln = all columns, Li ∩ Lj = primary key only.

Replication levels

Level	Reads / availability	Updates / complexity
Full (everywhere)	Best	Worst (every copy updated)
Partial (some copies)	Good	Moderate — the usual choice
None (one copy each)	Lowest	Simplest

Homogeneous vs heterogeneous (one line each)

- **Homogeneous:** same DBMS everywhere, sites cooperate, less autonomy, easy.
- **Heterogeneous:** different DBMSs/schemas, limited cooperation, more autonomy, hard. **Federated** = one global schema, minimal autonomy. **Multidatabase** = no global schema, full autonomy.

Distributed query processing: the 7 stages

Query mapping → Localization → Global optimization (communication cost dominates) → Local optimization → Join processing (ship query / ship data / hybrid) → Partial aggregation → Result merging.

Cost tricks: push σ and π to each site; aggregate locally; ship the **smaller** table; use a **semijoin** when few rows match.

Lecture numbers: EMPLOYEE 1,000,000 bytes, DEPARTMENT 3,500 bytes. Q best = 403,500 bytes (ship DEPARTMENT). Q′ best = 7,500 bytes. Semijoin for Q′ = 4,800 bytes.

Concurrency control and commit

Technique	Idea	Plus	Minus
Primary site	One coordinator site does all locking	Simple (like centralized 2PL)	Overload; single point of failure (use a backup site)
Primary copy	Lock the primary copy of each item; primary copies spread over sites	No single overloaded site	Need a directory to find primary copies
Voting	Ask all copies; a **majority** grants the lock	Truly distributed	More messages; complex with failures
2PC	Prepare/vote → commit only if all vote YES	All-or-nothing commit	Blocks if the coordinator fails
3PC	Adds a pre-commit phase	Non-blocking	More messages

Recovery: logs + checkpoints + distributed commit (2PC / 3PC) + coordinator election + bring recovered sites up to date before they rejoin.

How-to-answer recipes

Recipe 1: Scenario question — "features that make it distributed" and "benefits" ★

1. **Features** (what makes it a DDB), each tied to the scenario:
   - Multiple **sites** (name the cities) each with its own DBMS and data.
   - Connected by a **network** (a WAN between countries).
   - Logically **one database** with a **global schema** (all orders form one database).
   - A **DDBMS** giving **transparency** (a user in one city queries all data without knowing where it is).
   - **Fragmentation / replication**; **distributed queries and transactions**; **local autonomy** of each branch.
2. **Benefits**, each with a one-line example from the scenario: availability (one site fails, the others continue), performance (local data, parallel reports), global access, scalability (add a branch), disaster recovery, data-residency compliance, cost (commodity servers).

Recipe 2: CAP — "which properties to prioritize" / "trade-offs" ★

1. Define C, A, P in one line each.
2. Say **P is mandatory** (WAN between the sites will fail sometimes), so the real choice is **C vs A during a partition**.
3. **If CP:** what goes right (correct stock, money, no double-selling) and what goes wrong (a branch must refuse/delay requests during an outage → lost sales).
4. **If AP:** what goes right (every branch keeps working) and what goes wrong (copies diverge → the last item sold twice; needs reconciliation).
5. **Recommend a mix by data type** in a small table: transactional core (stock, payments, orders) → **CP**; read-mostly or tolerant data (catalogue, tracking, reports) → **AP / eventual consistency**.
6. **Reduce the impact:** fragment by region so most transactions are **local** to one site (no cross-site coordination needed); replicate read-only reference data; synchronous replication for critical data, asynchronous for the rest; quorum reads/writes.

Recipe 3: Fragmentation, replication and allocation strategy for a scenario ★

Use three headings, and give a **reason** for every choice:

1. **Fragmentation:**
   - Main transactional table (Orders) → **horizontal by branch/region**: σ branch='Colombo' (Orders), etc. Reason: each branch mostly uses its own orders → local, fast; keeps working if the WAN fails; rebuilt with UNION for global reports.
   - Child tables (OrderItems, Payments) → **derived horizontal**, following their order, so related rows are on the same site.
   - Customers → horizontal by home region; optionally **vertical** to keep sensitive columns (payment details) at the home site (each fragment keeps Customer_ID).
2. **Replication:**

- Read-heavy, rarely changed data (product catalogue, prices, branch list, exchange rates) → **fully replicated** at all sites.
- Orders → **not fully replicated** (too many inserts); primary copy at the home branch + **one backup copy** elsewhere (or at head office) for disaster recovery and reporting.

3. **Allocation table:** fragment → site(s).
4. **Summary:** maximizes locality, availability, low update cost, transparency for global queries, and data-residency rules.

Recipe 4: Distributed query processing questions

List the 7 stages with one line each, then the optimization tricks (push σ/π to sites, partial aggregation, ship the smaller relation, semijoin), then the challenges (communication cost, cost estimation with stale statistics, large search space, replica choice, failures during execution, skew, heterogeneity, security).

Recipe 5: Compare / define questions

Definition → comparison table → one example → one-line conclusion.

Lesson 5 slide exercises (short answers)

1. Study popular distributed databases (Cassandra, MongoDB, Couchbase, Google Spanner, Amazon DynamoDB) and discuss their design and architecture.

System	Data model	Architecture	CAP leaning
Cassandra	Wide-column	Peer-to-peer ring of equal nodes (no master); data spread by hashing the partition key; replicated to several nodes	**AP**, eventual consistency (consistency level can be tuned per query)
MongoDB	Documents (JSON-like)	**Replica sets** (one primary, several secondaries) + **sharding** by a shard key	Leans **CP** (writes go to the primary; a new primary is elected on failure)
Couchbase	Documents / key-value	Memory-first nodes, data spread over partitions, cross-data-centre replication	**AP** (lecture example)
Google Spanner	Relational (SQL)	Globally distributed; synchronous replication with consensus (Paxos) and synchronized clocks (TrueTime)	**CP**, strong global consistency
Amazon DynamoDB	Key-value / document	Fully managed; partitioned by a hash key; each item copied to several data centres in a region; global tables across regions	**AP** by default (eventual reads), strongly consistent reads optional

2. Explain the CAP theorem, how it influences design choices, with examples. See Lesson part 1, section 1.6, and the Cheat box. Summary: only two of C, A, P can be guaranteed; partitions are unavoidable, so systems choose **CP** (HBase, ZooKeeper, Spanner: correct data, may refuse requests) or **AP** (Cassandra, Couchbase, DynamoDB: always answer, eventually consistent). CA is only possible on a single site.

3. Key principles and challenges of recovery in distributed databases; logging, checkpointing, 2PC and 3PC.

- **Principle:** after any failure, return to a **consistent state** without losing committed data, and keep **all sites in agreement** about which transactions committed.
- **Challenges:** failures can happen at **any site or link**; changes are spread over several sites and copies; network partitions; a coordinator can fail in the middle of a commit.
- **Logging:** every change is written to a log first, so a site can **undo** unfinished and **redo** committed work.
- **Checkpointing:** save the state periodically so recovery does not replay the whole log (faster).
- **2PC:** prepare/vote then commit/abort → all sites commit or none (but can block if the coordinator fails).
- **3PC:** adds a pre-commit phase so participants can decide without the coordinator (non-blocking).
- Plus: coordinator election, backup sites, and bringing recovered sites up to date before they rejoin.

4. Elastic scalability in cloud-based distributed databases.

- **Meaning:** the system **adds or removes nodes automatically** as the workload goes up or down (scale out at peak, scale in when quiet), and you pay only for what you use.
- **How the cloud enables it:** virtual machines/containers started on demand, auto-scaling rules on CPU/traffic, managed services that **re-partition (re-shard) data** across new nodes, load balancers, storage separate from compute.
- **Challenges:** moving (rebalancing) data takes time and network; keeping data **consistent while it moves**; **hot partitions** (one key gets most traffic); cold-start delays; cost control; more complex monitoring.

5. Relational vs NoSQL distributed databases: scalability, fault tolerance, consistency.

	Relational (distributed SQL)	NoSQL
Data model	Tables with a fixed schema; SQL with joins	Key-value, document, wide-column, graph; flexible schema
Scalability	Traditionally **vertical**; horizontal scaling by sharding is harder (joins and transactions across shards)	Designed for **horizontal** scaling (add nodes, automatic partitioning)
Fault tolerance	Replication + failover; distributed commit (2PC)	Built-in replication to several nodes; often leaderless or automatic leader election
Consistency	**ACID**, strong consistency (leans CP)	Often **BASE**, eventual or tunable consistency (often AP)
Best for	Transactions needing correctness (banking, orders)	Huge volumes, high write rates, flexible data (social media, IoT, logs)

Past paper questions with answers

Grouped by topic. ★ = asked in more than one paper.

A. Concepts and architecture

> **2022/23 Q6(a) (25m) and 2023/24 Q7(c) (20m) ★** Explain the concept of a distributed database and how it differs from a centralized database. / Define a distributed database system, its key characteristics and architecture, and how it differs from a centralized system in data storage, processing and transparency.

Definition: a **distributed database (DDB)** is a collection of multiple, **logically related** databases **distributed over a computer network**. A **DDBMS** is the software that manages it and makes the distribution **transparent** to users. It consists of **loosely coupled sites** that share no physical components; a transaction may use data at one or several sites and is executed by them in a unified way.

Key characteristics:

- Data spread over **multiple sites** connected by a **network**; each site has its own DBMS, processor and storage.
- **Logically one database** (a global schema over all sites).
- **Fragmentation** (horizontal/vertical) and **replication** of data.
- **Transparency:** location, naming, fragmentation, replication; design and execution transparency.
- **Autonomy** of sites (design, communication, execution).
- **Distributed query processing and distributed transactions** (2PC); **reliability and availability** through replication and fault isolation.

Architecture: draw the diagram from Lesson part 1, section 1.2 (global users → global schema + DDBMS components → sites with local schema, DBMS and data → network).

Aspect	Centralized	Distributed
Data storage	All data at **one site**	Data **fragmented and/or replicated** over several sites, near its users
Processing	One DBMS runs every query	Queries **split into sub-queries** at several sites, run **in parallel**, results **merged**; communication cost matters
Transparency	Only logical/physical data independence	Must also give **distribution transparency** (users do not see locations, fragments, copies)
Reliability	Single point of failure	Failure isolated to one site; copies keep data available
Scalability	Vertical	Horizontal (add sites)
Transactions	Local concurrency control and recovery	Distributed locking, deadlock detection, **2PC**

Aspect	Centralized	Distributed
Complexity	Simpler	Complex (fragmentation, allocation, catalog, consistency)

> **2021/22 Q3(a) (25m).** With the aid of a diagram, explain the distributed database architecture.

Draw the diagram from **Lesson part 1, section 1.2**, then explain each part:

- **Sites (nodes):** independent computers, each with a local DBMS and database storing **fragments / copies**; loosely coupled, no shared memory or disks.
- **Communication network:** LAN or WAN connecting the sites; carries queries, data and results.
- **Global conceptual schema:** the single logical description users see.
- **Fragmentation schema:** how global tables are split (σ for horizontal, π for vertical).
- **Allocation schema:** which site(s) store each fragment (replication).
- **Local schemas:** how each site stores its part.
- **DDBMS components:** global query processing and optimization (decomposition, localization), distributed **transaction management** (concurrency control, 2PC, recovery), **replication** management, the **distributed catalog (directory)**.
- Styles: **client–server**, **peer-to-peer (collaborating servers)**, **federated / multidatabase**.
- The architecture gives **distribution transparency**: users query the global schema; the DDBMS maps the query to fragments and sites.

(The lecture shows only the "distributed database environment" figure; the layered schema architecture is standard textbook material.)

B. Scenario: features and benefits

> **2023/24 Q6(a) (25m).** A multinational company with branches in Colombo, London and Singapore plans to integrate their local customer order databases into a distributed database to improve availability, performance and global data access. Identify the features that make this a distributed database system, and explain the benefits the company can expect.

Features that make it a DDB:

1. **Multiple sites:** the Colombo, London and Singapore servers each keep their own database (local DBMS and storage).
2. **Connected by a network** (a WAN between continents).
3. **Logically related data under one global schema:** customer orders from all branches form **one logical database**.
4. **A DDBMS** coordinating the sites and giving **transparency**: a user in London can query all orders without knowing where they are stored.
5. **Fragmentation and replication** of the orders data across branches; **distributed queries and transactions** across sites.
6. **Local autonomy:** each branch keeps processing its local orders.

Benefits:

Benefit	For this company
High availability / reliability	If the Singapore server or link fails, Colombo and London keep working; copies keep data reachable; no single point of failure
Performance	Each branch reads its own orders **locally** (low delay); global reports run **in parallel** at three sites; load is shared
Global data access	Head office sees all orders as one database (location transparency)
Scalability	Add new branches as new sites without replacing hardware
Disaster recovery	Copies in other countries survive a local disaster
Local autonomy and compliance	Data can stay in its region (data-protection laws); local management
Cost	Ordinary servers instead of one huge central server
Easier development / expansion	Transparency hides distribution from applications

C. Design issues, system types, fragmentation and replication

> **2021/22 Q3(b) (25m).** Explain possible issues associated with distributed database design.

1. **Fragmentation:** choosing the units (horizontal, vertical, mixed); fragments must be **complete** and **reconstructable** (UNION / outer union or join), ideally **disjoint**.
2. **Allocation:** deciding which site stores each fragment, based on where and how often data is used; finding a good allocation is a **complex optimization problem**.
3. **Replication:** how many copies and where — more copies give better availability and reads, but **costly updates** and harder concurrency control and recovery.
4. **Distributed catalog / directory:** tracking fragments, copies and locations; where to keep the directory (global or local).
5. **Distributed query processing:** network (WAN) cost dominates; choosing strategies (which site does what, semijoins).
6. **Distributed transactions and concurrency control:** multiple copies, distributed locking (primary site / primary copy / voting), **distributed deadlock**.
7. **Recovery and reliability:** site failures, **link failures**, **network partitioning**, **distributed commit** (2PC / 3PC).
8. **Heterogeneity:** different DBMSs, data models, constraints, query languages.
9. **Security** across sites and networks; **autonomy** vs global control.
10. **Consistency vs availability** trade-off (CAP).

2021/22 Q3(c) (25m). Compare homogeneous and heterogeneous databases.

- **Homogeneous DDBS: all sites run the same DBMS** (e.g. all Oracle), are **aware of each other** and **agree to cooperate**; local sites give up some autonomy (cannot freely change schema or software); operating systems may differ.
- **Heterogeneous DDBS:** sites may use **different DBMSs and different schemas** (even data models); they may **not know each other** and offer **limited cooperation**. Schema differences complicate **query processing**; software differences complicate **transactions** across sites. Sub-types: **federated** (one global schema, minimal local autonomy) and **multidatabase** (no global schema, full autonomy). Heterogeneity comes from different **data models, constraints and query languages**.

	Homogeneous	Heterogeneous
DBMS software	Identical	Different
Schemas / data models	Common	May differ
Cooperation	Full	Limited
Local autonomy	Lower	Higher
Query processing	Straightforward	Translation and integration needed
Transactions	Standard protocols (2PC)	Difficult
Design / administration	Simpler	Complex (schema integration)
Typical origin	Built as a DDB	Integrating existing / legacy systems

2021/22 Q3(d) (25m). Describe data fragmentation and replication, with their benefits and drawbacks compared to a centralized database.

Fragmentation = splitting tables into fragments stored at different sites.

- **Horizontal:** subsets of rows, σ C (R) (e.g. EMPLOYEE where Dno = 5 at department 5's site); rebuilt with **UNION**. **Derived horizontal** fragmentation follows a related table's fragments.
- **Vertical:** subsets of columns, π L (R), **each including the primary key**; rebuilt with outer union / join on the key.
- **Mixed:** π L (σ C (R)).

Replication = storing copies of fragments at more than one site: **full** (everything everywhere), **partial**, or **none**.

	Benefits compared with a centralized DB	Drawbacks compared with a centralized DB
Fragmentation	**Data locality** (stored where used) → less network traffic and faster local access; **parallel** queries; smaller tables to scan; local control and security of sensitive data	Queries needing several fragments require **distributed joins/unions** and communication; constraints across fragments are harder; complex design; rebuilding costs time
Replication	**Higher availability and reliability** (no single point of failure); **faster reads** from a nearby copy; load sharing; disaster recovery	**Updates must reach all copies** (slow writes); **consistency**, concurrency control and recovery much more complex; more storage; risk of stale data
(Centralized DB)	Simple, always consistent, no internal communication	Single point of failure, slow for remote users, limited scalability

Lecture guidance: full replication suits high availability with mostly read-only transactions; data used mainly at one site should live only there; limit replication when updates are frequent.

> **2023/24 Q6(b) (25m) and Q7(d)(i) (15m) ★** Discuss suitable fragmentation and replication strategies for the customer orders database across Colombo, London and Singapore (or New York, London, Singapore), with justification.

Fragmentation:

- **Orders (and OrderItems): horizontal by branch:** Orders_Colombo = σ branch='Colombo' (Orders), and the same for the other cities. *Why:* each branch creates and mostly reads **its own** orders → fast local processing; branches keep working if the WAN fails; the fragments are **complete and disjoint** and are rebuilt with **UNION** for global reports.
- **OrderItems / Payments: derived horizontal**, following their order, so an order and its lines are on the same site (local joins).
- **Customers:** horizontal by home region; optionally **vertical** to separate sensitive personal/payment columns (kept at the home site because of data-protection laws) from general profile columns (both keep Customer_ID).

Replication:

- **Product catalogue, prices, branch list, exchange rates** (read-heavy, rarely updated) → **fully replicated** at all three sites → local reads, high availability.
- **Order fragments** → **not fully replicated** (constant inserts would have to be applied everywhere); keep the primary copy at the home branch plus **one backup copy** at another site or head office → disaster recovery and global reporting.
- Customers who buy from several branches → partial replication to those branches.

Allocation:

Fragment	Site(s)
Orders / OrderItems of branch X	Branch X (primary) + backup copy at a partner site

Fragment	Site(s)
Product catalogue, prices	All sites
Customers of region X	Region X (+ a copy where often used)
Global analytics	Head office / data warehouse (periodic loads)

Justification: most transactions stay on **one site** (low delay), sites keep working during network problems (**availability**), write-heavy data is not copied everywhere (**low update cost**), global queries still see one database (**fragmentation transparency**), and data stays in its country (data-residency rules).

D. CAP theorem and properties in scenarios ★

> **2022/23 Q6(b) (25m).** Discuss the CAP theorem and how it applies to distributed databases.

CAP theorem (Brewer's theorem): a distributed data store can guarantee **at most two** of:

- **Consistency (C):** every read returns the **most recent write**; all nodes see the same data at the same time.
- **Availability (A): every request gets a response** (read or write), even if some nodes are down, though the data may not be the latest.
- **Partition tolerance (P):** the system **keeps operating** when the network splits and some nodes cannot communicate.

Key insight: partitions cannot be avoided in real distributed systems, so **P must be provided**. So **during a partition** a system must choose between **C** (reject or delay some requests to avoid inconsistency) and **A** (answer every request, maybe with stale data). With no partition, both C and A are possible.

Types: CP (HBase, ZooKeeper) — may refuse requests during partitions. **AP** (Cassandra, Couchbase) — temporary inconsistency, **eventual consistency**. **CA** — only theoretical for distributed systems.

How it applies to DDB design:

- It comes from **replication**: copies at several sites must be kept in step over the network.
- Designers choose **per type of data**: banking / inventory → CP; social feeds, carts, catalogues → AP with eventual consistency.
- It is implemented through synchronous vs asynchronous replication, quorum/voting, distributed commit (2PC), conflict resolution.
- It explains why different database models (NoSQL systems tuned for C or for A) exist.
- *Example:* two bank branches lose their link: refuse withdrawals (CP) or allow them on possibly old balances (AP).

2022/23 Q6(c)(i) (25m). A distributed database for a multinational logistics company (warehouses, shipping routes, inventory across regions; large transaction volume, high availability, consistency across sites). Explain how the core properties of distributed databases affect its design and operation.

Property	Effect on this logistics system
Distribution (location / naming) transparency	Dispatchers anywhere query Inventory, Shipments and Routes without knowing which regional server holds them; data can be moved without changing applications; needs a **global catalog** with unique names
Fragmentation transparency	Inventory and shipment tables are **horizontally fragmented by region/warehouse** for local speed; a global query ("total stock of item X worldwide") still sees one table, so the DDBMS must localize and merge fragment queries
Replication transparency	Routes, product master data and warehouse lists are **replicated** to all regions for fast reads; the DDBMS keeps copies consistent (synchronously for critical data)
Autonomy	Each regional warehouse keeps receiving, picking and shipping when links to other regions fail; local maintenance is possible; global coordination rules must be agreed
Reliability and availability	One warehouse server failing does not stop the others; copies and backup sites keep data reachable 24/7; faults are isolated
Distributed transaction management	Inter-warehouse transfers update two sites **atomically (2PC)** so stock is never lost or counted twice; distributed concurrency control stops two regions from allocating the same stock
Distributed query processing	Global reports run in parallel at regional sites with partial aggregation; communication cost is minimized
Scalability	New regions or warehouses are added as new sites
Consistency vs availability (CAP)	Decides which data must be strongly consistent (stock levels) and which must stay available (tracking)

2022/23 Q6(c)(ii) (25m). For this logistics system, which CAP properties would be prioritized, and why?

- **Partition tolerance is a must:** regional data centres are linked over WANs; partitions will happen; the system must keep running.
- **Consistency for the transactional core → CP:** inventory levels, stock allocation, order and shipment status changes, billing. Inconsistent stock across sites causes **over-selling, the same pallet or truck allocated twice, failed deliveries and financial loss**, and the requirement explicitly asks for **consistency across sites**. During a partition, conflicting cross-region updates are blocked or queued rather than accepted inconsistently.
- **Availability for read-heavy, tolerant data → AP:** shipment tracking, vehicle locations, dashboards, route suggestions. Customers and drivers must always get an answer; a few

seconds of staleness is acceptable (**eventual consistency**).

- **Reduce the impact:** fragment data so most transactions are **local to one region** (each region stays available and consistent for its own data during a partition); replicate read-only reference data; synchronous replication within a region, asynchronous across regions for non-critical data.
- **Conclusion: CP** (with P) for the inventory/order core, **AP** for tracking and analytics — a balance that meets both the consistency and the availability requirements.

> **2023/24 Q6(d) (25m).** Using the CAP theorem, analyse the trade-offs the company (Colombo, London, Singapore) must consider when designing the distributed database.

1. **Partitions will happen** between Colombo, London and Singapore (intercontinental WAN) → **P is required**; the real choice is **C vs A during a partition**.
2. **If they choose consistency (CP):** order, stock and payment data stay correct everywhere; but during an outage a branch may have to **reject or delay** orders that need other sites' data → lost sales and unhappy customers.
3. **If they choose availability (AP):** every branch keeps accepting orders during outages; but copies may **diverge** (the last unit sold twice, conflicting customer updates) → they need **reconciliation**, conflict-resolution rules, refunds or back-orders after the network recovers.
4. **Delay even without partitions:** strong consistency across continents needs **synchronous** coordination (2PC / quorums), adding long round trips to every write; asynchronous replication is fast but temporarily inconsistent.
5. **Decide per type of data:**

Data	Choice	Reason
Stock levels, payments, order confirmation	**CP**	Correctness and money
Product catalogue, prices	**AP** (replicated everywhere)	Read-mostly; brief staleness is acceptable
Order history, reporting	**AP / eventual**	Analytics can tolerate delay

6. **Reduce the trade-off:** fragment horizontally by branch so most orders are **local** (no cross-site coordination); quorum reads/writes for shared data; clear conflict-resolution rules; monitor partition events.

E. Distributed query processing and transactions

> **2023/24 Q6(c) (25m).** Customer order reports often need data from several locations (Colombo, London, Singapore). Explain how distributed query processing and optimization are performed, and the main challenges.

How it is done (lecture stages):

1. **Query mapping:** the report query (e.g. monthly sales per product across all branches) is written against the **global schema** and translated into an algebraic query on global tables; normalized, checked, simplified.
2. **Localization:** global tables are replaced by their **fragments**: Orders → Orders_Colombo ∪ Orders_London ∪ Orders_Singapore; replicated tables (Products) are read from the nearest copy.
3. **Global optimization:** choose the cheapest strategy, where **communication cost usually dominates** (WAN). Techniques:
   - push **selections and projections** to each site (only this month, only needed columns);
   - **partial aggregation** at each site (local SUM/COUNT per product), shipping only small summaries;
   - **ship the smaller relation** to the larger one's site (lecture: ship DEPARTMENT, 3,500 bytes, not EMPLOYEE, 1,000,000 bytes);
   - **semijoins**: send only join-column values to filter remote rows before shipping them (lecture: 4,800 bytes vs 1,000,000 for Q').
4. **Local optimization** at each branch (indexes, local join algorithms).
5. **Join processing** (ship query to data / data to query / hybrid), **aggregation** and **result merging** at the result site (union the partial results, combine sums and counts).

Main challenges:

- **High communication cost and delay** over intercontinental links.
- **Cost estimation is harder:** statistics of remote fragments may be out of date; network speed varies.
- **Huge number of strategies** (where to run each step, join order, semijoins).
- **Choosing a copy** and making sure it is not stale.
- **Site or link failures** during the query; partial results.
- **Uneven load / skewed data** between branches; **heterogeneity** (different DBMSs or schemas); **security** of data in transit; currency and time-zone differences in reports.

> **2023/24 Q7(d)(ii) (15m).** (New York, London, Singapore.) Explain how query processing and transaction management would ensure consistency and efficiency across the sites.

Query processing (efficiency):

- Global queries are split and **localized** to the branch fragments; selections and aggregations pushed to the sites; small replicated tables read locally; semijoins and "ship the smaller relation" reduce transfers; parallel execution and merging at the result site.
- Most everyday queries are **local** to one branch's fragment → no network cost.

Transaction management (consistency):

- **Local transactions** (orders of the local branch) commit locally.
- **Distributed transactions** (e.g. an order using stock from another branch) are coordinated by the originating site with **two-phase commit**: *prepare* (all participants vote) → *commit / abort* (commit only if all vote yes), so all sites commit or none. **Three-phase commit** avoids blocking if the coordinator fails.

- **Concurrency control:** distributed two-phase locking with a **primary copy** scheme (lock the primary copy of each replicated item) or **voting (majority)**; **distributed deadlock** detection or time-outs.
- **Keeping copies consistent:** synchronous updates for critical data (stock, payments); asynchronous for catalogue data.
- **Recovery:** logs and checkpoints at each site; **coordinator election** or a backup site if the coordinator fails; recovering sites catch up before rejoining.

F. Fault tolerance

> **2022/23 Q7(f) (25m, short note).** Fault tolerance in distributed databases.

Fault tolerance = the ability of a distributed database to **keep working correctly despite failures** of sites, disks or communication links. A **fault** causes an **error** (a bad state), which may cause a **failure** (the system does not behave as specified). **Reliability** = probability the system is up at a given moment; **availability** = probability it stays available over a period.

Types of failure in a DDB: site (node) crashes, disk failures, **communication link failures**, **network partitioning**, failures during **distributed commit**.

Mechanisms:

1. **Replication / redundancy:** copies at several sites; if one fails, others serve requests.
2. **No single point of failure / fault isolation:** a failure stays at its site; other sites continue.
3. **Primary site with a backup site**, **primary copy**, and **coordinator election** when a coordinator fails.
4. **Voting (majority / quorum):** a majority of copies can still grant locks and serve data when some are unreachable.
5. **Distributed commit protocols: 2PC** (all-or-nothing), **3PC** (non-blocking if the coordinator crashes).
6. **Logging and checkpointing** to recover each site to a consistent state; recovered sites are **brought up to date** before rejoining.
7. **Handling partitions:** the CAP trade-off — stay consistent (CP) or available (AP, eventual consistency).
8. **Geographic distribution and disaster recovery:** copies in other regions survive a site-wide disaster.
9. Inside each site: **RAID** and backups.

Importance: high availability for 24/7 global services and protection against data loss — but replication and coordination add **update cost and complexity**.

Questions NOT in the mid scope

Past paper questions on Object-Relational databases, Data Warehousing/OLAP, the three-schema architecture and NoSQL are left out, because the mid covers Lessons 1–5 only.